Software Testing

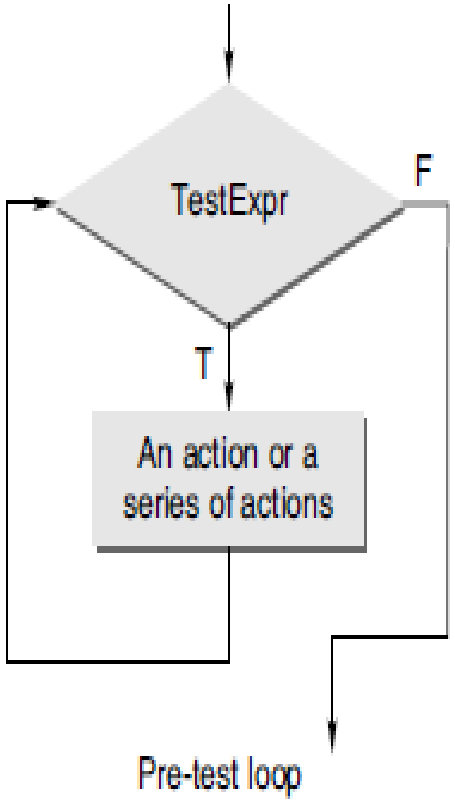
Draw the control flow graph, compute cyclomatic complexity and basis paths

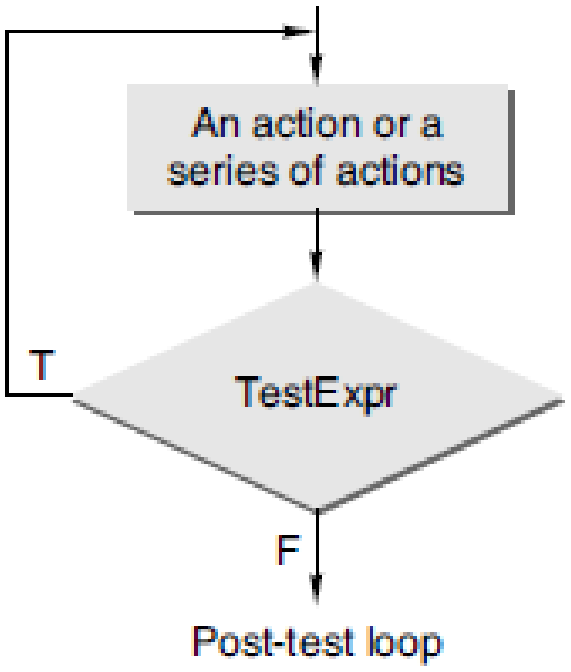
```
#include <stdio.h>
int main()
{
  int x;
  printf("\n ENTER THE NUMBER:");
  scanf("%d", &x);
  if(x > 0)
    printf("x is positive \n");
  else if(x == 0)
       printf("x is zero \n");
     else
       printf("x is negative \n");
  return 0;
}
```

Draw the control flow graph, compute cyclomatic complexity and basis paths

```
#include <stdio.h>
int main()
{
  int n;
  printf("\n Enter the number:");
  scanf("%d",  &n);
  switch(n%2)
  {
    case 0: printf("\n EVEN");

        break;
      case 1: printf("\n ODD");
        break;
    }
    return 0;
  }
```

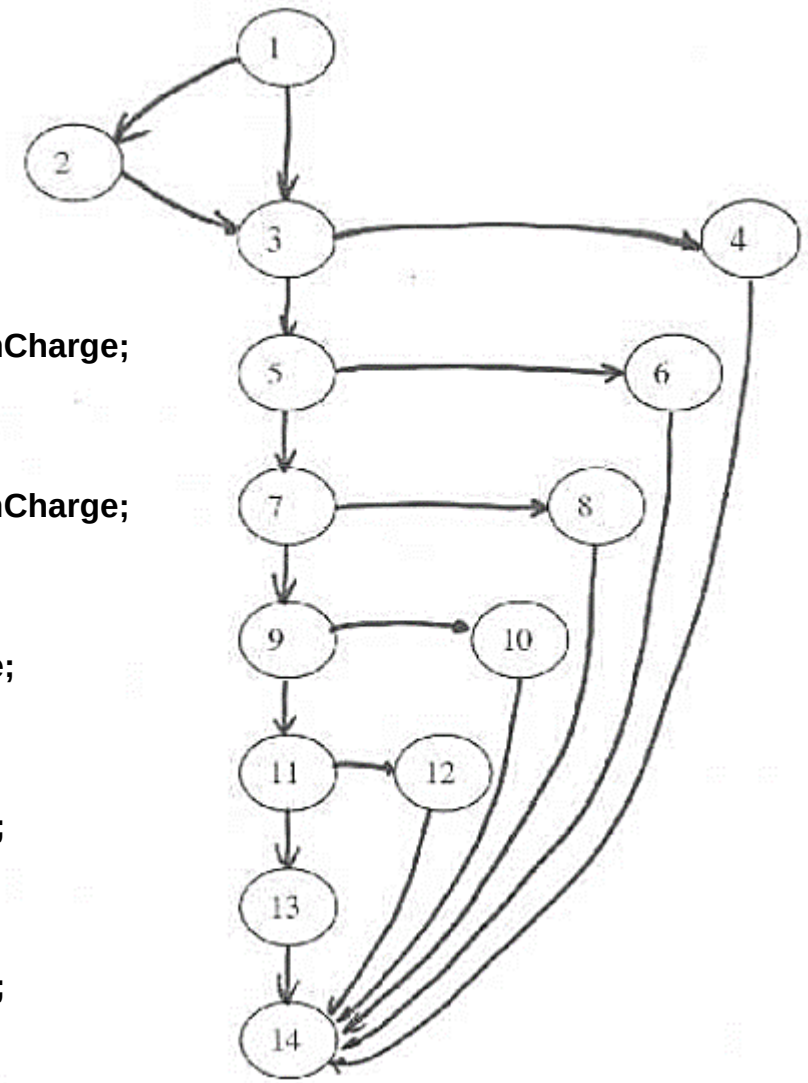





```
#include <stdio.h>
int main()
{
  int times = 0;
  while (times < 50)
  {
    printf( "*");
    times++;
  }
  return 0;
}
```

```
int main()
{
  int n, c;
  long int f=1;
  printf("\n Enter the number: ");
  scanf("%d",&n);
  for(c=1;c<=n;++c)
    f*=c;
  printf("\n Factorial  is %ld",f);
  return 0;
}
```

```
public double calculate(int amount)
  {
-1-  double rushCharge = 0;
-1-  if (nextday.equals("yes") )
    {
-2-      rushCharge = 14.50;
    }
-3-  double tax = amount * .0725;
-3-  if (amount >= 1000)
    {
-4-      shipcharge = amount * .06 + rushCharge;
    }
-5-  else if (amount >= 200)
    {
-6-      shipcharge = amount * .08 + rushCharge;
    }
-7-  else if (amount >= 100)
    {
-8-      shipcharge = 13.25 + rushCharge;
    }
-9-  else if (amount >= 50)
    {
-10-     shipcharge = 9.95 + rushCharge;
    }
-11- else if (amount >= 25)
    {
-12-     shipcharge = 7.25 + rushCharge;
    }
    else
    {
-13-     shipcharge = 5.25 + rushCharge;
    }
-14- total = amount + tax + shipcharge;
-14- return total;
    } //end calculate
```



Step 2: Determine the cyclomatic complexity of the flow graph.

$$V(G) = E - N + 2$$
$$= 19 - 14 + 2$$
$$= 7$$

Step 3: Determine the basis set of independent paths.

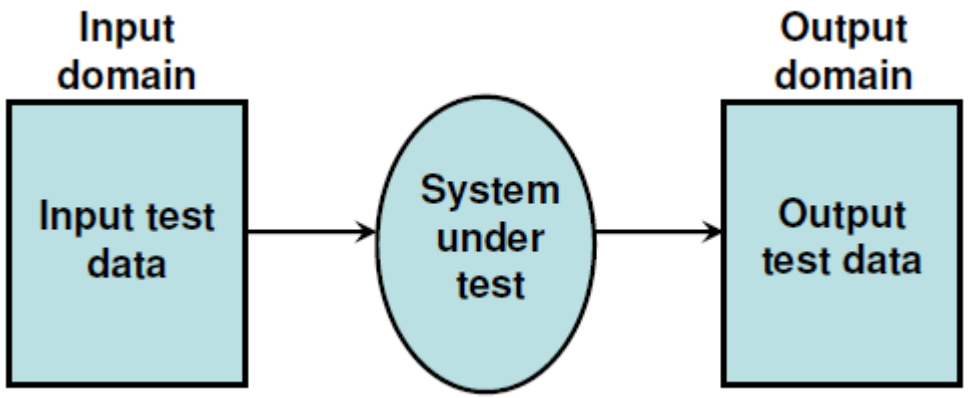
Path 1: 1 - 2 - 3 - 5 - 7 - 9 - 11 - 13 - 14
Path 2: 1 - 3 - 4 - 14
Path 3: 1 - 3 - 5 - 6 - 14
Path 4: 1 - 3 - 5 - 7 - 8 - 14
Path 5: 1 - 3 - 5 - 7 - 9 - 10 - 14
Path 6: 1 - 3 - 5 - 7 - 9 - 11 - 12 - 14
Path 7: 1 - 3 - 5 - 7 - 9 - 11 - 13 - 14

Write a program that checks whether a character entered by the user is a vowel or not

- Draw the control flow graph, compute cyclomatic complexity and basis paths

Black Box Testing

- *Black-box testing,* also called *behavioral testing,* focuses on the functional requirements of the software. That is, black-box testing enables the software engineer to derive sets of input conditions that will fully exercise all functional requirements for a program.
- Black-box testing is not an alternative to white-box techniques. Rather,it is a complementary approach that is likely to uncover a different class of errors than white-box methods.



Black-box testing attempts to find errors in the following categories:

1. incorrect or missing functions
2. interface errors,
3. errors in data structures or external database access,
4. behavior or performance errors, and
5. initialization and termination errors.

- Unlike white-box testing, which is performed early in the testing process, blackbox testing tends to be applied during later stages of testing

Types of Black Box Testing

1. Equivalence Partitioning
2. Boundary Value Analysis
3. Cause Effect Graphing

Equivalence Partitioning

Principles

- An exhaustive testing of values in the input domains is impossible.
- One is limited to a small subset of all possible input values.
- One wants to select a subset with the highest probability of finding the most errors.

Consider a program that calculates the root of quadratic equations in the form of:

$$a x^2 + b x + c = 0$$

with the solution for the root to be:

$$r = (-b \pm \sqrt{b^2 - 4 a c}) / (2 a).$$

If each variable is represented by a 32 bit floating point number, the number of all possible input value combinations is then

$$2^{32} \times 2^{32} \times 2^{32} = 2^{96}.$$

Equivalence Classes

- A well-selected set of input values should covers a large set of other input values.
- This property implies that one should partition the input domains into a finite number of equivalence classes.
- A test of a representative value of each class is equivalent to a test of any other value.

Valid and Invalid Equivalence Classes

- The equivalence classes are identified by taking each input condition and partitioning the input domain into two or more groups.
- Two types of equivalence classes are identified.
- Valid equivalence classes represent valid inputs to the program.
- Invalid equivalence classes represent all other possible states of the condition.

`

If an input condition specifies a range of values (e.g., the count can be from 1 to 999), it identifies one valid equivalence class (1 ≤count ≤999) and two invalid equivalence classes (count < 1and count > 999)